

⇒ int '0x21'

- Dos has entry point through int 21h
- Service number is provided in 'ah'

Application

↓

OS

↓

BIOS

↓

Hardware.

mov ax, 0x4C00 // 4C means termination service
int 21h // this is a parameter for int 21h call which will

To use any service terminate program.

through service number in

ax (in DOS), we'll always write 21h after 'int'.

⇒ Input Service:

mov ah, 01h

int 21h

→ service number for inputting a character.

* 01h will let us pressed keyboard for input.

The key pressed's ASCII will be stored in 'al' to be used in program.

mov ax, 0xB800

mov es, ax

mov ~~ah~~ ^{ah}, 1 // read character from keyboard & save in 'al'

int 0x21

mov [es:0], al // displays the key pressed on keyboard on 1st row 1st column.



* Always: DI updated with ES: DI & SI with DS: SI



* cld → DF = 0 (increment 1/2)
* std → DF = 1 (decrement 1/2)



Part of CHAP: 7 "STRINGS"

	Source	Destination	
<u>STOS:</u>	• AL/AX → ES: DI • updates DI • rep (cx times)	reg → video/str	
<u>LODS:</u>	• DS: SI → AL/AX • updates SI • loop (cx times), not rep	video/str → reg	
<u>SCAS:</u> ZF	• AL/AX ↔ ES: DI • updates SI/DI • repe/repne (dec cx)	reg ↔ video/str	
<u>MOVS:</u>	• DS: SI → ES: DI • updates SI & DI • rep	str → str str → video X not possible as video memory require an attribute as well which isn't present in str: db 'Hello'.	
<u>CMPS:</u> flags changed (like ZF)	• DS: SI ↔ ES: DI • updates SI & DI • repe/repne	video/str ↔ video/str	

* Video memory movement & comparison done after we're done "mov ES: DI, 0xB800".

* we can also do both string move/comparison by moving data into ES: DI.



LECTURE - 18 Thursday 30/10/25

Services of int:

⇒ Output string:

mov ah, 0x09 → parameter for int 21h to print a string
int 21h

* DS & DX are also passed as parameter to ISR where DS: DX will have a "\$ terminated string".

Code for outputting a string:

```
[org 0x0100]
mov ah, 9 // service 9h for outputting string in 'dx'
mov dx, str1
int 0x21
```

```
mov ax, 0x4C00
```

```
int 0x21
```

```
str1: db 'abcde $' // $ terminated string
```

low level service (not Dos OS service).

⇒ BIOS Keyboard Service: Works better than int 21h as it also tells scan code

```
mov ah, 0
```

```
int 16h
```

Returns ASCII of ↑ keyboard in 'al' and scan code of key pressed in 'ah' character pressed on

like ASCII. For every key on keyboard, there is scan code. Some keys like arrows don't have ASCII.

low level (not Dos service)

⇒ BLOS video services: display, set video mode, set cursor, shape, read/write at cursor position, scroll etc.

int 10h

↓ whenever this interrupt is called; the data from following registers will be fetched as parameter for string print etc.

ah = 0x13 (service which prints string): one of service of int 10h)

al = write mode. al = 1 means update cursor)

bh = Page number

bl = Attribute

dh = row number

dl = column number

cx = length of string

es = segment of string

bp = offset of string (arrow pointing each character of string).

→ upon this; dl will keep incrementing to printed next character in next column till end of dhth row. After that dh will increment for next row.

Code for String printing using "int 10":

mov ah, 0x13

mov al, 1

mov bh, 0

mov bl, 7 // white text on black screen bg.

mov dx, 0x0A03 // row: 10, column: 3.

mov cx, 11

push cs

pop es

mov bp, message

int 0x10

mov ax, 4C00h

int 0x21

message: db 'meow meow cat' // No '\$' terminated need since length in cx.

⇒ INT 0: Division overflow (automatically called when division by zero).

⇒ INT 1: This is generated after every instruction to see if trap flag is 1 or not.

⇒ INT 3: Debug interrupt (for breakpoints)

⇒ INT 4: Arithmetic overflow (manually done but done as a result of special instr. INTO)

⇒ INT 10h: BLOS video service

⇒ INT 16h: BLOS keyboard services

⇒ INT 21h: DOS services.

- ISR is simply a func. that do a specific thing when called jisko ham interrupt keh rahay hotay (like: taking input).
- These func. are mapped by different numbers like "int 9h". The track record of these func. are in IVT. Processor gets to (9*4)th index of IVT when "int 9h" called and there it finds the

Where is ISR located? ^{segment & offset no. jahan actual ISR wali func. paray hai fir 9h.}

- At physical address 00000h, there is interrupt vector table (IVT).

- IVT table contains the physical addresses of ISRs of each interrupt in segment: offset form.

- Each entry in IVT is of 4 bytes. First two bytes ~~of ISR~~ ^{are} of segment of ISR and next two bytes of offset of ISR.

For ^{nth} interrupt;

offset = $n \times 4$ Byte

segment = $(n \times 4) + 2$ Byte

decimal value not Hexd.

Bytes	
0-1	offset int 0
2-3	segment int 0
4-5	offset int 1
6-7	segment int 1
8-9	offset int 2
10-11	segment int 2

offset of int '2' = $2 \times 4 = 8$

⇒ INTERRUPT VECTORING:

- CPU processes the INT instruction using the IVT.
- Steps computer takes when INT n is invoked by program:
 - Operand of INT (i.e. n) is multiplied by 4 to locate matching IVT entry
 - The CPU pushes FLAGS, CS, IP (return address) on the stack and disables interrupt & trap flags.

simplified code in class comments of "Software Interrupt" GCR lecture.

* Never use BIOS/DOS services like: int 21h, int 10h inside definition of custom ISR while ↑ int.

- CPU loads seg. of interrupt from IVT to CS ($n \times 4 + 2$) offset to IP ($n \times 4$)
- Interrupt handler executes until IRET is called.
- The IRET instruction pops flags and return address from the stack and execution is resumed from the next instruction where the int instruction was written.

⇒ Hooking an Interrupt: Means to change the vector corresponding to an interrupt in IVT.

- * For calling our custom written routine for any specific interrupt (e.g. print a message when division interrupt)

⇒ Code for custom routine for 'Int 80'

mov ax, 0

mov es, 0

// as IVT is on zeroth segment

mov word [es: 80 * 4], custom ISR // store offset at $n \times 4$.

mov word [es: 80 * 4 + 2], CS // store 'cs' segment at $n \times 4 + 2$.

Now whenever 'int 80h' will be called; our ISR i.e. Custom ISR will be called instead of default routine.

NOTE: While hooking an interrupt; we're changing the entire functionality of that interrupt with ours (not just updating). But doing `jmp far [cs: old_isr]` instead of `iret` brings specific interrupt back to IVT key (80×4)th index pr ↑ function ka offset hai; uss function ko "custom ISR" say replace krdo. The original func. of that isr which we didn't considered while customizing.

LECTURE-19 Thursday 8/11/25

Hardware Interrupts:

- Events that occur outside the processor (like: mouse, keyboard etc).
- Occurs when a key is pressed by user.
- Asynchronous and unpredictable (key could be pressed at any time).
- Single pin outside processor is called INT pin is used by hardware to generate interrupts.
- Since multiple hardware and only one INT pin therefore a controller called "Programmable Interrupt Controller" (8259 PIC) is used to schedule the interrupt (priority schedule).

⇒ PIC: 8 input signals & one output signal.

- Assigns priority to eight of input pins (0→7):
 - 0 has highest priority & 7 has lowest
 - lower level interrupt cannot be handled before higher prio.
- Highest priority is forwarded to INT (output pin) & rest are on hold.

This means that the interrupt func. i.e: ISR is located on the segment & offset which is in the IVT in the $(8 \times 4)^{th}$ index.

⇒ IRQ: Interrupt Requests.

- These are 8 input signals send to PIC
- Each IRQ is mapped onto a specific interrupt system.

Table in slides:

IRQ	Interrupt	Description
0	8h	System timer
1	9h	Keyboard
⋮	⋮	⋮
7	0Fh	LPT1

Priority (Keyboard) > priority (LPT1).

in output string: ah = 9h

these aren't the 'ah' value like and int = 21h.

Example: A key press on keyboard generates IRQ 1 and INT 9 is invoked which stores the key.

- When processor receives a hardware interrupt's number from PIC, it pushes flags, CS, IP, read IVT to get ISR of interrupt & execute it.
- At the end of servicing the interrupt, the handler should inform PIC that it's completed, so lower priority interrupts could be handled now.
- This signal of informing PIC is called "End of Interrupt" (EOI) and is sent through I/O ports of PIC.

mod ah, 20h

out 20h, ah

1:31 pm

8/11/25

diagram gap here points

LECTURE-20

Saturday
8/11/25

I/O Ports: ^{memory addresses: like CPU will know} keyboard trying to interact when 0x60 address is used with 'in'/'out' means port 60.

- Use to communicate with external devices.
- Each device has 16 bit port numbers/address (means 2^{16} I/O ports available)
- A pin is set on control bus to show that the communication is to be between I/O device and CPU
- lower 16 bits of address bus is used to specify port number.
- Example 20 & 21 port numbers are for PIC; 60 and 64 for keyboard, 378 for parallel port.

⇒ IN / OUT INSTRUCTION:

- Used to read and write data to ports (just like "mov" used to read & write in memory).
- IN : Instruction to read from a port
- OUT : " " write to a port.

Example:

- To read data from keyboard: ^{mostly fixed} in al, 0x60.
- When key pressed; the scan code of key from keyboard is sent to ^{always} 0x60 ^{memory}. The above instruction will take the address of port from "0x60" in memory and store scan code in 'al'.

• 'in' is to get any hardware device onboard or trapped (eg: keyboard, "in al, 0x21" is for mask register). Then after that; we do customisation on 'al' to get our desired output. And then we use "out __, al" to inject these custom changes back into that hardware in blank spaces ('out' doesn't do direct work).

mov dx, 0x378

// address of port saved in dx.

in al, dx

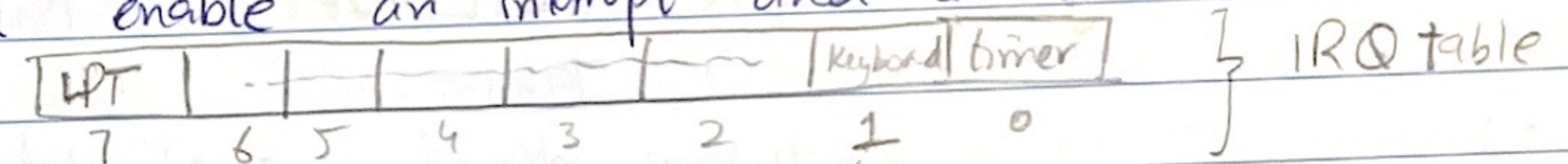
// moves the port no. from the address saved in dx (does not directly copy data)

out dx, al

// outputs the byte in port no. that is in 'al'.

⇒ **PIC PORTS**: PIC has two ports → 20 & 21.

- Port 20 is control part
- Port 21 is the interrupt mask register which can be used for selectively enabling or disabling interrupts
- '0' will enable an interrupt and '1' bit will disable.



⇒ Disabling Keyboard using PIC/PORT 21:

in al, 0x21

// reads (interrupt from 21h of PIC

or al, 2

// Set bit two (of keyboard) by MASK.

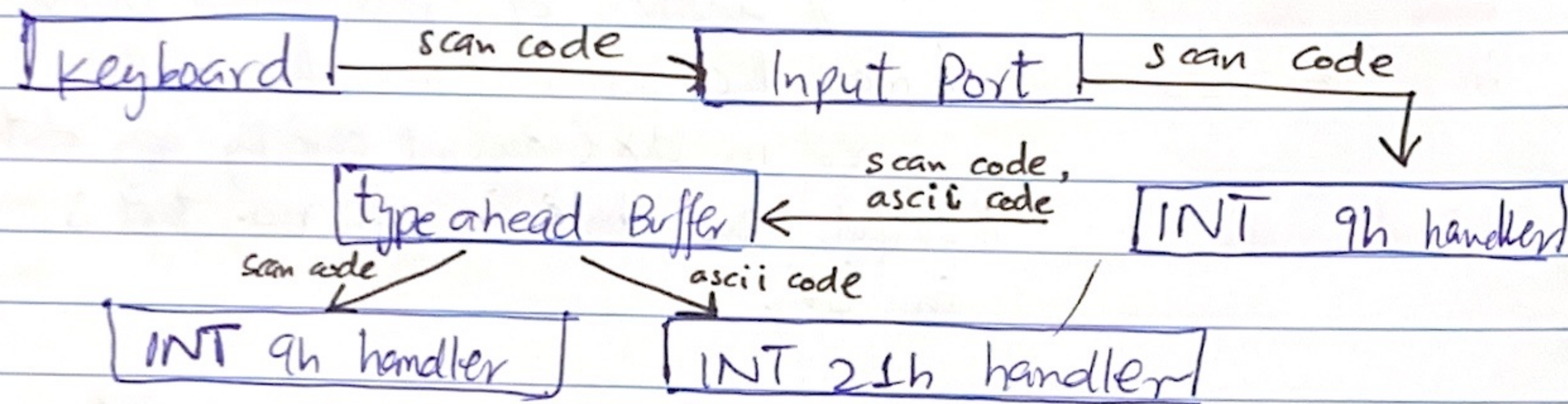
out 0x21, al

// write back masked value into 21h of PIC which disabled keyboard;

as the second bit is turned off now.

How keyboard works?

- Int 9h is hardware interrupt generated when a key is "pressed" or "released"
- For each key, the scan code comes twice; once for the key press and once for key release. ^{differ only by one bit}
- Both the scan code of press and released
- lower 7 bits contain key no. The MSB contains zero in press code and 1 when released.



LECTURE-21 Tuesday 11/11/23

Hooking Hardware INTERRUPT: To change ISR address in INT table.

- Before hooking a hardware interrupt, we should make sure that no hardware interrupt does not come in b/w hooking instruction.
- ∴ we use **CLI** to disable all hardware interrupt & **STI** after instruction of hooking.
- Before hooking hardware interrupt shld be zero & after hooking, it shld be one.

Example: Hooking keyboard Interrupt:

Q: Assembly code to print 'L' when 'leftshift' pressed & prints 'R' when 'rightshift' pressed.

[org 0x0100]
jmp Start

KbISR:

push ax
push es
mov ax, 0xB906
mov es, ax

in al, 0x60

cmp al, 0x2a // '2a' is ASCII of Left Shift

jne nextcmp

mov byte [es:0], 'L' // always prints at top left

jmp nonmatch

nextcmp:

cmp al, 0x36 // '36' is ASCII of Right Shift

jne nonmatch

mov byte [es:0], 'R'

nonmatch:

mov al, 0x20 // 0x20 contains entire keyboard (default)
out 0x20, al // and it will run like usual.

pop es
pop ax
iret

NOTE: • Since we ran our custom ISR $\therefore$ only left & right shift key will work; rest keyboard is disable.
• Restart dosBox to re-initialise the IVT & enable it.

Solution

$\Rightarrow$ INTERUPT CHAINING: So we can run original ISR as well as our custom one to use usual keyboard as key as the custom keys (L/R Shift we made on prev page).

- Save original ISR in the IVT in a variable before interrupt hooking.
- Remove EOI code and iret instruction in our custom ISR and add the following instruction: `jmp far [cs: oldisr]` $\rightarrow$ variable

`jmp Start`
`KbISR:`

```
pushax
push es
mov ax, 0xB800
mov es, ax
in al, 0x60
cmp al, 0x2a
jne nextcmp
mov byte [es:0], 'L'
jmp nonmatch
```

`Start:`

```
mov ax, 0
mov es, ax // points es to IVT
mov ax, [es: 9*4]
mov [oldisr], ax
mov ax, [es: 9*4 + 2]
mov [oldisr + 2], ax
cli // disables interrupts
mov word [es: 9*4], KbISR // store
offset at n*4.
mov [es: 9*4 + 2], cs // store
segment at n*4 + 2
sti // enables interrupts
mov ax, 0x4C00
```

next cmp:

```
cmp al, 0x36
jne nonmatch
mov byte [es:0], 'R'
nonmatch:
```

```
mov al, 0x20 // commented
out 0x20, al // commented
```

`pop es`

`pop ax`

`iret`

`jmp far [cs: oldisr]`

far jump since the "KbISR" we made is suppose in segment A but the original ISR from IVT lies in segment B. $\therefore$ we need to switch segments to reach oldisr, so 'far'

$\Rightarrow$ Terminate ~~Stay~~ Resident: If we run another program; it will run after current program is terminated. It won't overwrite the current program. Use:

```
mov ax, 0x3100
int 21h
```

for termination instead of 0x4C00

we need to save code in these bytes.

```
mov dx, start // 'start' contains no. of Bytes from start till main
add dx, 15 // Round up.
mov cl, 9 // divides cb by '16' to convert in paragraph
mov ax, 0x3100 as one para. is of 16 bytes.
int 21h
```

LECTURE-22

Thursday
12/11/25.

☐ Unhooking Interrupts: Reset the interrupt vector to the value it had before the start of our program

Code to unhook interrupt when ESC pressed: (continue of left start: right Shift keys)

```

mov ax, 0
mov es, ax // points es to IRT base
mov ax, [es:9*4]
mov [oldisr+2], ax
cli // disables interrupt
mov word [es:9*4], 0 // stores offset at nx4
mov [es:9*4+2], 0 // stores segment at nx4+2
sti // enables interrupts
b1: mov ah, 0 // service 0 - get keystroke
int 0x16 // call BIOS Keyboard service
cmp al, 27 // is the ESC key pressed
jnc b1 // if no, check for next key
mov ax, [oldisr] // read old offset in ax
mov bx, [oldisr+2] // read old segment in bx
cli // disables interrupts
mov [es:9*4], ax // restore old offset from ax
mov [es:9*4+2], bx // restore old segment from bx.
sti
mov ax, 0x400 // terminate
int 0x21.
    
```

☐ Selective Chaining:

☐ Programmable Interval Time^(PIT): Peripheral device.

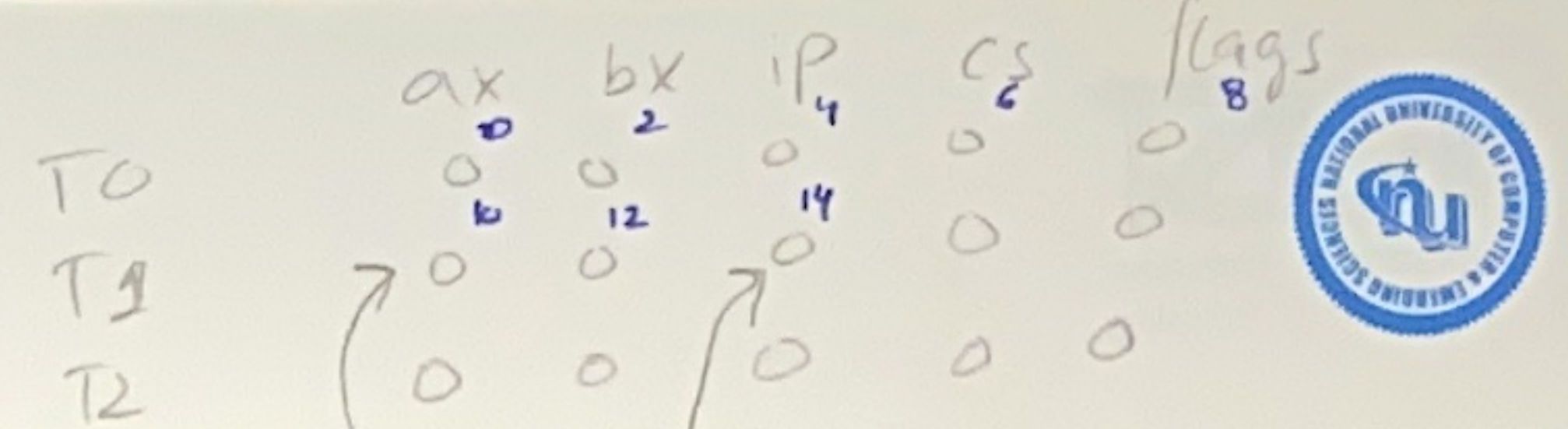
- A timer device built with an aim to resolve the time control issue occurring in blue various processes occurring within the micro-processor
- Used to maintain system time

⇒ PIT Interrupt:

- Inside the chip, there is 16-bit divisor that divides the input freq (i.e. 1.193 MHz)
- The output is connected to the IRQ 0 of the PIC
- The special number 0 if placed in the divisor means a divisor of 65536 and not 0.
- By default, IRQ 0 is generated 18.2 times per second i.e. after every 55 ms.

$$(1.19 \times 10^6) / 65536 = 18.2$$

→ changing this in 40h port will fast or slow timer
- Highest Priority Interrupt it is.
- IRQ 0 is mapped to interrupt 8h
- we can change the 16-bit divisor to change freq. of interrupt
- 16-bit divisor is placed on port 40h.



⇒ Code to Change 16-Bit Divisor:

```
mov ax, [PIT_reload_value] // ax=16 bit value loaded
out 0x40, al // set low byte of PIT reload value
mov al, ah // ax=high 8 bits of reload value
out 0x40, al // set high byte of PIT reload value.
```

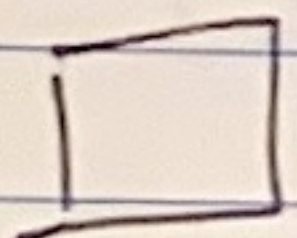
→ explicitly (we don't call it).

Everytime IRQ 0 is ranned
means 18 times in second;

LECTURE-23 Tuesday
18/11/25.

tickcount variable is incremented & displayed on video memory

⇒ Code for Printing Counter } In Slides.
⇒ Code for Printing Counter with keyboard }



CHAPTER "MultiTasking".

LECTURE-24 Thursday
20/11/25.

→ Rest from Slides & code in GCR.

⇒ Implement MultiTasking:

- When timer interrupt is called; the return address, flags and CS is pushed are pushed in stack so handler could return back to his point when interrupt ends.
- But for multitasking; we need to go from Task 1 → interrupt Task 2 → interrupt → Task 3 instead of Task 1 → interrupt → Task 1.
- For moving handler to next task instead of back to previous task; we'll change values of "return address, CS, Flags and assign it with next task's values. After changing these values in stack; we'll call int.

start: mov word [taskstates+10+4], taskone
This ~~interru~~ saves the 'ip' (return address) of taskone in the 2nd row third column of array (taskstates)

mov word [es: 8*4], timer
Timer interrupt is gonna save the flags, cs, registers & IP of current task and is gonna run the next task.

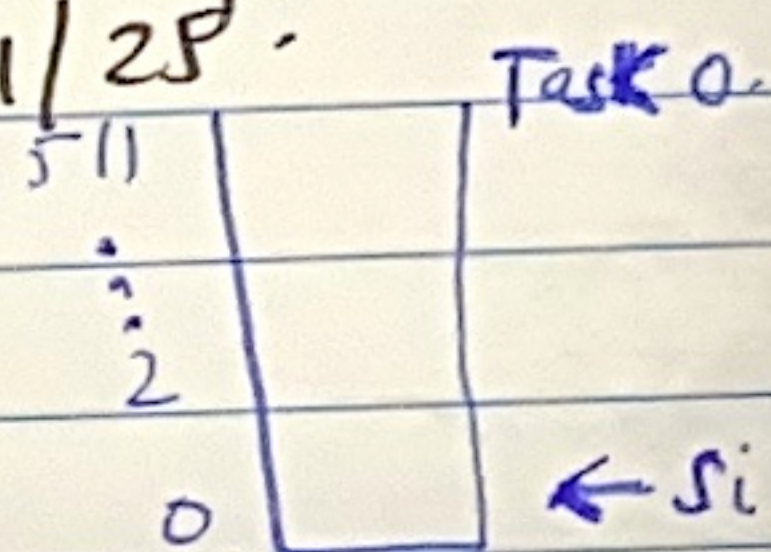
mov bl, [cs: current] // current is the variable containing current task being run (0, 1, 2). "cs:current" as we always use 'cs' instead of 'ds' to access variables in interrupt.

mov word [taskstates+10+8], 0x200. // initialise flags (4th index)
It is 0x200 bcs in flag register; it will only set the interrupt flag = 1 and rest all flags = 0.

LECTURE-25 Tuesday
25/11/25.

Stack: (ss) variable

0 - 511 Bytes for Task 0
512 - 1023 Bytes for Task 1



★ Code in Slides.

⇒ Safe termination: two parts ① & ②.

① Interrupt chaining: end the custom hooked isr with ^{not only} "jmp far [cs:old_isr]" instead of isr so we can use our custom isr first but also the original isr after our custom ones. eg: customising only 3 keys in keyboards & then doing chaining to restore functionality of other keys.

② Unhooking interrupts: we changed the IRT with our custom 'kb_isr' being placed in it in the beginning. so we need to return our original isr which CPU has everytime the same back to IRT & we do it by returning old_isr to specific int n index we moved kb_isr. This returns the old segment no. & offset to the IRT as "old_isr" was dd (2 bytes for seg & 2 for offset).